ULC 301

DATA STRUCTURES

BY: SHIVNEET KAUR
1024040074

3D13 (ELE)

Q1.Write a program in C to convert miles into kilometers (Km). Hint: 1 Mile=1.609 Km. [Use macros, relevant name and types for variables].

SOL:

```
#include <stdio.h>


#define MILE_TO_KM 1.609   // Conversion factor


int main() {
    double miles, kilometers;


    printf("Enter distance in miles: ");
    scanf("%lf", &miles);


    kilometers = miles * MILE_TO_KM;


    printf("%.2f miles = %.2f kilometers\n", miles, kilometers);


    return 0;
```

}
OUTPUT:

```
Enter distance in miles: 5
5.00 miles = 8.05 kilometers

Enter the number of inputs: 6
Enter 6 numbers:
10 -3 0 5 -7 0
Positive numbers: 2
Negative numbers: 2
Zeros: 2
```

Q2.Write a program to find the number of positive, negative and zeros in a sequence of inputs (numbers) entered as data.
SOL:

```
#include <stdio.h>


int main() {
    int n, num;
    int positive = 0, negative = 0, zeros = 0;


    printf("Enter the number of inputs: ");
    scanf("%d", &n);


    printf("Enter %d numbers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &num);
        if (num > 0)
            positive++;
        else if (num < 0)
            negative++;
        else
            zeros++;
    }
```

```
    printf("Positive numbers: %d\n", positive);
    printf("Negative numbers: %d\n", negative);
    printf("Zeros: %d\n", zeros);

    return 0;
}
```

OUTPUT:

```
Enter the number of inputs: 6
Enter 6 numbers:
10 -3 0 5 -7 0
Positive numbers: 2
Negative numbers: 2
Zeros: 2
```

Q3.

Compute the tax due based on a tax table given below:
Program Input: Salary amount.
Program Output: Returns the tax due for 0.0 <= salary <= 150,000.00; returns -1.0 if salary is outside the table range.

Salary Range ($)	Base Tax ($)	Percentage of Excess
0.00–14,999.99	0.00	15
15,000.00–29,999.99	2,250.00	18
30,000.00–49,999.99	5,400.00	22
50,000.00–79,999.99	11,000.00	27
80,000.00–150,000.00	21,600.00	33

SOL:

```
#include <stdio.h>

int main() {
    int n, num;
    int positive = 0, negative = 0, zeros = 0;
```

```
    printf("Enter the number of inputs: ");
    scanf("%d", &n);

    printf("Enter %d numbers:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &num);
        if (num > 0)
            positive++;
        else if (num < 0)
            negative++;
        else
            zeros++;
    }

    printf("Positive numbers: %d\n", positive);
    printf("Negative numbers: %d\n", negative);
    printf("Zeros: %d\n", zeros);

    return 0;
}
```

OUTPUT:

```
Enter the number of inputs: 6
Enter 6 numbers:
10 -3 0 5 -7 0
Positive numbers: 2
Negative numbers: 2
Zeros: 2
```

Q4. 4) Write an interactive program (menu driven) in ‘C’ (using functions) to compute the area of a selected geometrical figure from a list of such figures (square, rectangle, and circle).

SOL:

```
#include <stdio.h>
#include <math.h>

double areaSquare(double side) { return side * side; }
double areaRectangle(double length, double width) { return length * width; }
double areaCircle(double radius) { return M_PI * radius * radius; }

int main() {
    int choice;
    double side, length, width, radius;

    do {
        printf("\n--- Area Calculator ---\n");
        printf("1. Square\n2. Rectangle\n3. Circle\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter side: ");
                scanf("%lf", &side);
                printf("Area of Square = %.2f\n", areaSquare(side));
                break;
            case 2:
                printf("Enter length and width: ");
                scanf("%lf %lf", &length, &width);
                printf("Area of Rectangle = %.2f\n", areaRectangle(length, width));
                break;
            case 3:
```

```
            printf("Enter radius: ");
            scanf("%lf", &radius);
            printf("Area of Circle = %.2f\n", areaCircle(radius));
            break;
        case 4:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice!\n");
    }
  } while(choice != 4);

  return 0;
}
```

OUTPUT:

```
--- Area Calculator ---
1. Square
2. Rectangle
3. Circle
4. Exit
Enter your choice: 1
Enter side: 5
Area of Square = 25.00
```

Q5. Write a program to display first n elements of Fibonacci series.

SOL:

```
#include <stdio.h>

int main() {
  int n, i;
  unsigned long long t1 = 0, t2 = 1, next;

  printf("Enter number of terms: ");
```

```
    scanf("%d", &n);

    printf("Fibonacci Series: ");
    for (i = 1; i <= n; i++) {
        printf("%llu ", t1);
        next = t1 + t2;
        t1 = t2;
        t2 = next;
    }
    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter number of terms: 7
Fibonacci Series: 0 1 1 2 3 5 8
```

Q6. Write a program to print a table book from Table X to Table Y. X and Y are user inputs.

SOL:

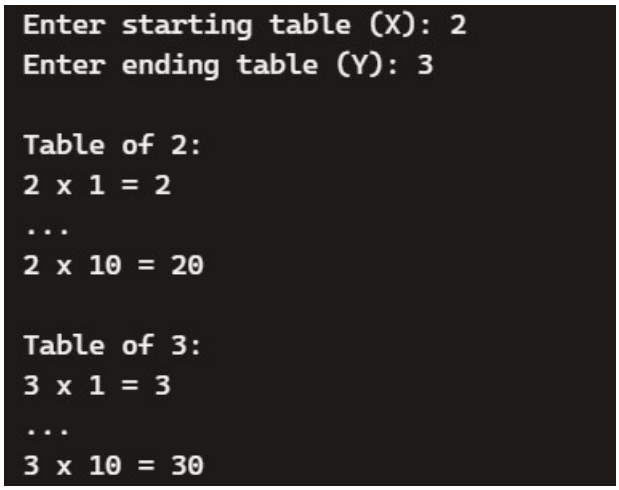
```
#include <stdio.h>


int main() {
    int X, Y, i, j;
    printf("Enter starting table (X): ");
    scanf("%d", &X);
    printf("Enter ending table (Y): ");
    scanf("%d", &Y);

    for (i = X; i <= Y; i++) {
        printf("\nTable of %d:\n", i);
        for (j = 1; j <= 10; j++) {
```

```
            printf("%d x %d = %d\n", i, j, i*j);
        }
    }
    return 0;
}
```

OUTPUT:



Q7. Write a program to compute factorial of a number using iterative approach.
SOL:

```
#include <stdio.h>

int main() {
    int n, i;
    unsigned long long fact = 1;

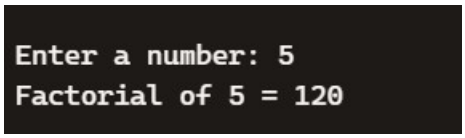
    printf("Enter a number: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        fact *= i;
    }

    printf("Factorial of %d = %llu\n", n, fact);
```

```
    return 0;
}
```

OUTPUT:



Q8. Write a program to swap two numbers using functions.
SOL:

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x, y;
    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);

    printf("Before swap: x=%d, y=%d\n", x, y);
    swap(&x, &y);
    printf("After swap: x=%d, y=%d\n", x, y);

    return 0;
}
```

OUTPUT:

```
Enter two numbers: 10 20
Before swap: x=10, y=20
After swap: x=20, y=10
```

Q9. Write a function that returns the first integer between n_min and n_max entered as data to the calling function (main).

SOL:

```
#include <stdio.h>


int getFirst(int n_min, int n_max) {
    return n_min;
}


int main() {
    int n_min, n_max;
    printf("Enter n_min and n_max: ");
    scanf("%d %d", &n_min, &n_max);


    printf("First integer between %d and %d is: %d\n", n_min, n_max, getFirst(n_min, n_max));
    return 0;
}
```

OUTPUT:

```
Enter n_min and n_max: 5 12
First integer between 5 and 12 is: 5
```

Q10.

Write nests of loops that cause the following output to be displayed.

```
0
0 1
0 1 2
0 1 2 3
0 1 2 3 4
0 1 2 3 4 5
0 1 2 3 4
0 1 2 3
0 1 2
```

SOL:

```
#include <stdio.h>


int main() {
    int i, j;

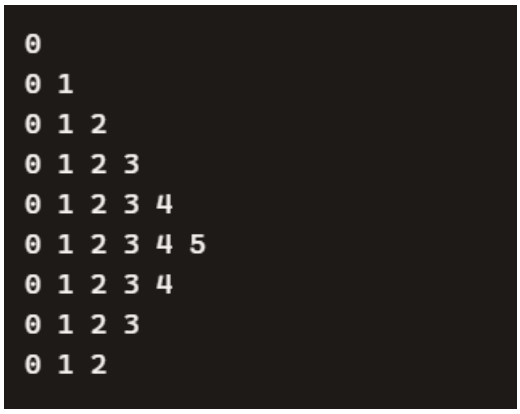

    // Upper half of the triangle
    for (i = 0; i <= 5; i++) {
        for (j = 0; j <= i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }


    // Lower half of the triangle
    for (i = 4; i >= 2; i--) {
        for (j = 0; j <= i; j++) {
            printf("%d ", j);
        }
        printf("\n");
    }


    return 0;
```

}

OUTPUT:



LAB ASSIGNMENT 2

1. Write a program to check whether a given number is present in an array or not (Linear search).

Sol:

```
#include <stdio.h>

int main()
{
    int a[100], n, i, key, found = 0;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    for(i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }
```

```
    if(found == 0)
        printf("Element not found\n");

    return 0;
}
```

```
Enter number of elements: 4
Enter array elements:
12 6 81 42
Enter element to search: 81
Element found at position 3


=== Code Execution Successful ===
```

2. Write a program to get second maximum and second minimum elements in an array.
Sol:

```
#include <stdio.h>
#include <limits.h>

int main()
{
    int a[100], n, i;
    int max1 = INT_MIN, max2 = INT_MIN;
    int min1 = INT_MAX, min2 = INT_MAX;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
for(i = 0; i < n; i++)
{
    if(a[i] > max1)
    {
        max2 = max1;
        max1 = a[i];
    }
    else if(a[i] > max2 && a[i] != max1)
    {
        max2 = a[i];
    }

    if(a[i] < min1)
    {
        min2 = min1;
        min1 = a[i];
    }
    else if(a[i] < min2 && a[i] != min1)
    {
        min2 = a[i];
    }
}

if(max2 == INT_MIN || min2 == INT_MAX)
    printf("Second maximum/minimum does not exist.\n");
else
{
    printf("Second Maximum = %d\n", max2);
    printf("Second Minimum = %d\n", min2);
```

```
    }

    return 0;
}
```

```
Enter number of elements: 4
Enter array elements:
21 15 71 7
Second Maximum = 21
Second Minimum = 15


=== Code Execution Successful ===
```

3. Write a program to perform insertion (any location), deletion (any location) and traversal in an array.

Sol:

```
#include <stdio.h>


int main()
{
    int a[100], n, i, pos, value, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    do
    {
        printf("\n--- MENU ---\n");
```

```
printf("1. Insertion\n");
printf("2. Deletion\n");
printf("3. Traversal\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
   case 1:
      printf("Enter position for insertion: ");
      scanf("%d", &pos);

      printf("Enter value: ");
      scanf("%d", &value);

      if(pos < 1 || pos > n + 1)
      {
         printf("Invalid position\n");
      }
      else
      {
         for(i = n; i >= pos; i--)
            a[i] = a[i - 1];

         a[pos - 1] = value;
         n++;

         printf("Element inserted successfully.\n");
```

```
    }
    break;

case 2:
    printf("Enter position for deletion: ");
    scanf("%d", &pos);

    if(pos < 1 || pos > n)
    {
        printf("Invalid position\n");
    }
    else
    {
        for(i = pos - 1; i < n - 1; i++)
            a[i] = a[i + 1];

        n--;

        printf("Element deleted successfully.\n");
    }
    break;

case 3:
    printf("Array elements are:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    break;
```

```
            case 4:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice\n");
        }

    } while(choice != 4);

    return 0;
}
```

```
Enter number of elements: 4
Enter array elements:
12 23 45 61

--- MENU ---
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 1
Enter position for insertion: 2
Enter value: 21
Element inserted successfully.

--- MENU ---
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice:
```

4. Write a menu driven program to perform addition, multiplication and subtraction of 2 arrays.

Sol: #include <stdio.h>

```
int main()
{
    int a[100], b[100], result[100];
    int n, i, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements of first array:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter elements of second array:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &b[i]);

    do
    {
        printf("\n--- MENU ---\n");
        printf("1. Addition\n");
        printf("2. Multiplication\n");
        printf("3. Subtraction\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
```

```
case 1:
    printf("Addition: ");
    for(i = 0; i < n; i++)
    {
        result[i] = a[i] + b[i];
        printf("%d ", result[i]);
    }
    printf("\n");
    break;

case 2:
    printf("Multiplication: ");
    for(i = 0; i < n; i++)
    {
        result[i] = a[i] * b[i];
        printf("%d ", result[i]);
    }
    printf("\n");
    break;

case 3:
    printf("Subtraction: ");
    for(i = 0; i < n; i++)
    {
        result[i] = a[i] - b[i];
        printf("%d ", result[i]);
    }
    printf("\n");
    break;
```

```
            case 4:
                printf("Exiting program...\n");
                break;

            default:
                printf("Invalid choice\n");
        }

    } while(choice != 4);

    return 0;
}
```

```
Enter number of elements: 3
Enter elements of first array:
12 7 81
Enter elements of second array:
12 42 72

--- MENU ---
1. Addition
2. Multiplication
3. Subtraction
4. Exit
Enter your choice: 2
Multiplication: 144 294 5832

--- MENU ---
1. Addition
2. Multiplication
3. Subtraction
4. Exit
Enter your choice: 3
Subtraction: 0 -35 9

--- MENU ---
1. Addition
2. Multiplication
3. Subtraction
4. Exit
Enter your choice: 1
Addition: 24 49 153
```

5. Write a program to perform sorting while merging (Merge two sorted arrays into one sorted array)

Sol: #include <stdio.h>

int main()

{

int a[100], b[100], c[200];

int n1, n2, i, j, k;

```
printf("Enter number of elements in first sorted array: ");
scanf("%d", &n1);

printf("Enter elements:\n");
for(i = 0; i < n1; i++)
   scanf("%d", &a[i]);

printf("Enter number of elements in second sorted array: ");
scanf("%d", &n2);

printf("Enter elements:\n");
for(i = 0; i < n2; i++)
   scanf("%d", &b[i]);

i = 0;
j = 0;
k = 0;

while(i < n1 && j < n2)
{
   if(a[i] < b[j])
      c[k++] = a[i++];
   else
      c[k++] = b[j++];
}

while(i < n1)
   c[k++] = a[i++];
```

```
    while(j < n2)
        c[k++] = b[j++];

    printf("Merged sorted array:\n");

    for(i = 0; i < k; i++)
        printf("%d ", c[i]);

    printf("\n");

    return 0;
}
```

```
Enter number of elements in first sorted array: 3
Enter elements:
12 3 34
Enter number of elements in second sorted array: 3
Enter elements:
21 56 12
Merged sorted array:
12 3 21 34 56 12


=== Code Execution Successful ===
```

6. Write the above programs (1,2, and 3) using functions and call by address only.

Sol: 6(a). Linear Search Using Function and Call by Address

```
inclu#de <stdio.h>

void linearSearch(int *a, int *n, int *key)
{
    int i, found = 0;

    for(i = 0; i < *n; i++)
    {
```

```
        if(*(a + i) == *key)
        {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
        }
    }

    if(found == 0)
        printf("Element not found\n");
}

int main()
{
    int a[100], n, i, key;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    linearSearch(a, &n, &key);

    return 0;
```

}

```
========================================
        LINEAR SEARCH
========================================

Enter number of elements: 5

Enter array elements:
10 20 30 40 50

Enter element to search: 40

Element found at position 4

========================================
```

6(b). Second Maximum and Second Minimum Using Function

```
#include <stdio.h>
#include <limits.h>

void secondMaxMin(int *a, int *n)
{
   int i;
   int max1 = INT_MIN, max2 = INT_MIN;
   int min1 = INT_MAX, min2 = INT_MAX;

   for(i = 0; i < *n; i++)
   {
      if(*(a + i) > max1)
      {
         max2 = max1;
         max1 = *(a + i);
      }
      else if(*(a + i) > max2 && *(a + i) != max1)
      {
```

```
            max2 = *(a + i);
        }

        if(*(a + i) < min1)
        {
            min2 = min1;
            min1 = *(a + i);
        }
        else if(*(a + i) < min2 && *(a + i) != min1)
        {
            min2 = *(a + i);
        }
    }

    printf("Second Maximum = %d\n", max2);
    printf("Second Minimum = %d\n", min2);
}

int main()
{
    int a[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
    secondMaxMin(a, &n);

    return 0;
}
```

```
=======================================
   SECOND MAXIMUM & SECOND MINIMUM
=======================================

Enter number of elements: 6

Enter array elements:
25 10 45 30 5 40

Second Maximum = 40
Second Minimum = 10

=======================================
```

6(c). Insertion, Deletion and Traversal Using Functions and Call by Address

```
#include <stdio.h>

void insertion(int *a, int *n, int *pos, int *value)
{
    int i;

    for(i = *n; i >= *pos; i--)
        *(a + i) = *(a + i - 1);

    *(a + *pos - 1) = *value;
    (*n)++;
}

void deletion(int *a, int *n, int *pos)
{
    int i;
```

```
    for(i = *pos - 1; i < *n - 1; i++)
        *(a + i) = *(a + i + 1);

    (*n)--;
}

void traversal(int *a, int *n)
{
    int i;

    printf("Array elements are:\n");

    for(i = 0; i < *n; i++)
        printf("%d ", *(a + i));

    printf("\n");
}

int main()
{
    int a[100], n, i, choice, pos, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter array elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
```

```
do
{
    printf("\n--- MENU ---\n");
    printf("1. Insertion\n");
    printf("2. Deletion\n");
    printf("3. Traversal\n");
    printf("4. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch(choice)
    {
        case 1:
            printf("Enter position: ");
            scanf("%d", &pos);

            printf("Enter value: ");
            scanf("%d", &value);

            insertion(a, &n, &pos, &value);
            printf("Element inserted successfully.\n");
            break;

        case 2:
            printf("Enter position: ");
            scanf("%d", &pos);

            deletion(a, &n, &pos);
```

```
            printf("Element deleted successfully.\n");
            break;

        case 3:
            traversal(a, &n);
            break;

        case 4:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid choice\n");
    }

} while(choice != 4);

return 0;
```

}

```
Enter number of elements: 5
Enter array elements:
17 2 61 45 7

--- MENU ---
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice: 2
Enter position: 4
Element deleted successfully.

--- MENU ---
1. Insertion
2. Deletion
3. Traversal
4. Exit
Enter your choice:
```

LAB ASSIGNMENT 3

String Processing operations

1) Write a program to implement strlen() function.

```
#include <stdio.h>


int my_strlen(char *str) {
    int len = 0;
    while (str[len] != '\0')
        len++;
    return len;
}


int main() {
    char str[] = "Hello World";
    printf("Length of string: %d\n", my_strlen(str));
    return 0;
}
```

Output:

```
Length of string: 11
```

2) Write a program to implement strcpy() function.

```
#include <stdio.h>


void my_strcpy(char *dest, char *src) {
    while (*src) {
        *dest = *src;
        dest++;
        src++;
    }
```

```
    *dest = '\0';
}

int main() {
    char src[] = "Hello";
    char dest[20];
    my_strcpy(dest, src);
    printf("Copied string: %s\n", dest);
    return 0;
}
```

Output:

```
Copied string: Hello
```

3) Write a program to implement strcat() function.

```
#include <stdio.h>

void my_strcat(char *dest, char *src) {
    while (*dest) dest++;
    while (*src) {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0';
}

int main() {
    char str1[30] = "Hello";
    char str2[] = " World";
```

```
    my_strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
    return 0;
}
```

Output:

```
Concatenated string: Hello World
```

4) Write a program to implement strcmp() function.

```
#include <stdio.h>

int my_strcmp(char *s1, char *s2) {
    while (*s1 && (*s1 == *s2)) {
        s1++;
        s2++;
    }
    return *(unsigned char*)s1 - *(unsigned char*)s2;
}

int main() {
    char str1[] = "Apple";
    char str2[] = "Apples";
    int result = my_strcmp(str1, str2);
    if (result == 0)
        printf("Strings are equal\n");
    else if (result < 0)
        printf("str1 < str2\n");
    else
        printf("str1 > str2\n");
    return 0;
```

}
Output:

```
str1 < str2
```

5) WAP to demonstrate limitations of Two-Dimensional Array of Characters.

```
#include <stdio.h>


int main() {
    char names[3][10] = {"Alice", "Bob", "Christopher"}; // Christopher truncated!
    for (int i = 0; i < 3; i++) {
        printf("Name[%d]: %s\n", i, names[i]);
    }
    return 0;
}
```

Output:

```
Name[0]: Alice
Name[1]: Bob
Name[2]: Christop
```

6) WAP to demonstrate an array of Pointers to Strings.

```
#include <stdio.h>


int main() {
    char *names[] = {"Alice", "Bob", "Christopher"};
    for (int i = 0; i < 3; i++) {
        printf("Name[%d]: %s\n", i, names[i]);
    }
    return 0;
}
```

Output:

```
Name[0]: Alice
Name[1]: Bob
Name[2]: Christopher
```

LAB ASSIGNMENT 4 (a)

Linked-list

1) Write a program to implement single Link List (with function insertion at first node, insertion at last node, insertion and deletion at middle node and traversing of link list after each node insertion).

```
#include <stdio.h>

#include <stdlib.h>

struct Node {

    int data;

    struct Node* next;

};


struct Node* head = NULL;


// Function to traverse and print the list

void traverse() {

    struct Node* temp = head;

    printf("Linked List: ");

    while (temp != NULL) {

        printf("%d -> ", temp->data);

        temp = temp->next;

    }

    printf("NULL\n");

}


// Insert at beginning

void insertFirst(int value) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    newNode->next = head;
```

```
    head = newNode;
    traverse();
}

// Insert at end
void insertLast(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    traverse();
}

// Insert at middle (after a given position)
void insertMiddle(int value, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;

    struct Node* temp = head;
    for (int i = 1; i < pos && temp != NULL; i++) {
```

```
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("Position not found!\n");
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    traverse();
}

int main() {
    insertFirst(10);
    insertLast(20);
    insertLast(30);
    insertMiddle(25, 2);
    insertFirst(5);
    return 0;
}
```

Output:

```
Linked List: 10 -> NULL
Linked List: 10 -> 20 -> NULL
Linked List: 10 -> 20 -> 30 -> NULL
Linked List: 10 -> 20 -> 25 -> 30 -> NULL
Linked List: 5 -> 10 -> 20 -> 25 -> 30 -> NULL
```

2) Write a program to implement single Link List (with function deletion at first node, deletion at last node, and deletion at middle node and traversing of link list after each node

deletion).

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void traverse() {
    struct Node* temp = head;
    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Insert helper for testing
void insertLast(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
```

```
    } else {
        struct Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

// Delete first node
void deleteFirst() {
    if (head == NULL) return;
    struct Node* temp = head;
    head = head->next;
    free(temp);
    traverse();
}

// Delete last node
void deleteLast() {
    if (head == NULL) return;
    if (head->next == NULL) {
        free(head);
        head = NULL;
    } else {
        struct Node* temp = head;
        while (temp->next->next != NULL) {
            temp = temp->next;
        }
```

```
        free(temp->next);
        temp->next = NULL;
    }
    traverse();
}

// Delete middle node (at given position)
void deleteMiddle(int pos) {
    if (head == NULL) return;

    struct Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Position not found!\n");
        return;
    }

    struct Node* delNode = temp->next;
    temp->next = delNode->next;
    free(delNode);
    traverse();
}

int main() {
    insertLast(10);
    insertLast(20);
```

```
    insertLast(30);
    insertLast(40);
    traverse();

    deleteFirst();
    deleteLast();
    deleteMiddle(2);

    return 0;
}
```

Output:

```
Linked List: 10 -> 20 -> 30 -> 40 -> NULL
Linked List: 20 -> 30 -> 40 -> NULL
Linked List: 20 -> 30 -> NULL
Linked List: 20 -> NULL
```

LAB ASSIGNMENT 5

Stacks and its Applications

1. Write a menu driven program with 4 options (Push, Pop, Display, and Exit) to demonstrate the working of stacks using arrays.

```
#include <stdio.h>

#define MAX 5


int stack[MAX], top = -1;


void push() {
    int val;
    if (top == MAX - 1)
        printf("Stack Overflow!\n");
    else {
        printf("Enter value to push: ");
        scanf("%d", &val);
        stack[++top] = val;
    }
}

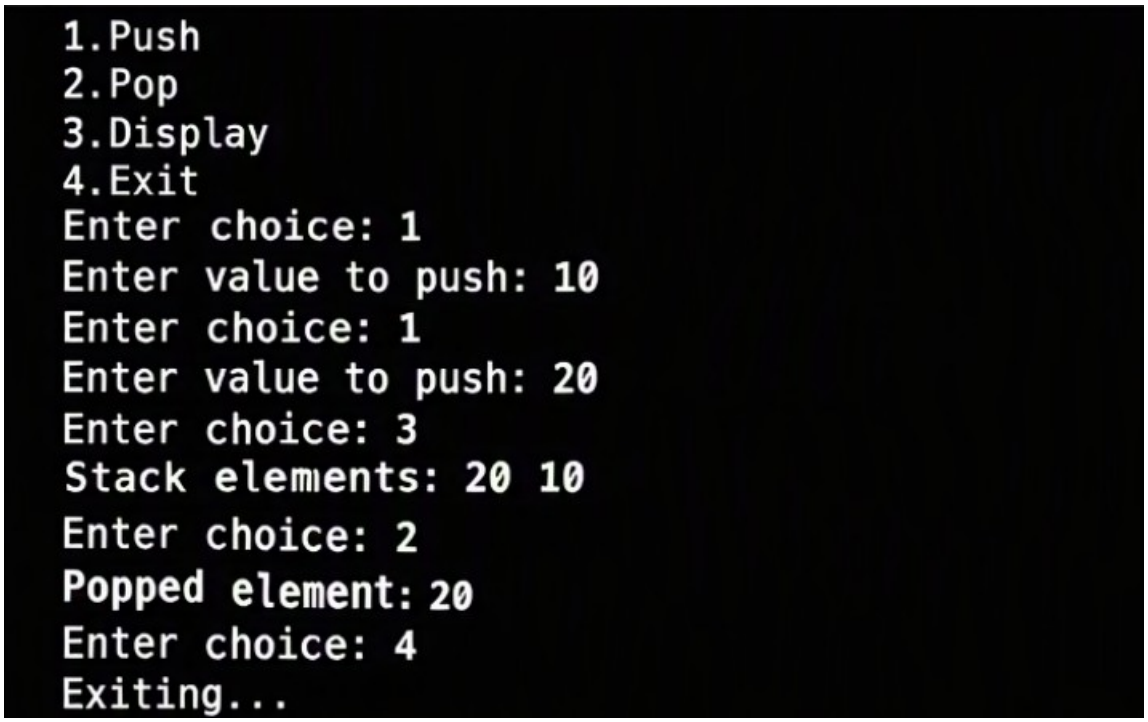

void pop() {
    if (top == -1)
        printf("Stack Underflow!\n");
    else
        printf("Popped element: %d\n", stack[top--]);
}


void display() {
    if (top == -1)
        printf("Stack is empty!\n");
```

```
    else {
        printf("Stack elements: ");
        for (int i = top; i >= 0; i--)
            printf("%d ", stack[i]);
        printf("\n");
    }
}

int main() {
    int choice;
    do {
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

Output:



2. Write a menu driven program with 4 options (Push, Pop, Display, and Exit) to demonstrate the working of stacks using linked-list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* top = NULL;

void push() {
    int val;
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    printf("Enter value to push: ");
```

```
    scanf("%d", &val);
    newNode->data = val;
    newNode->next = top;
    top = newNode;
}

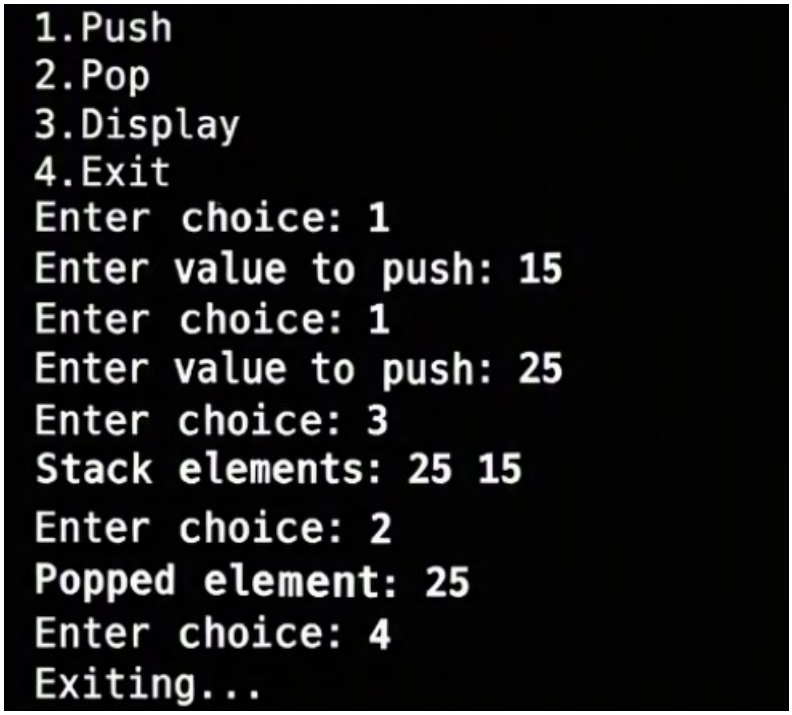
void pop() {
    if (top == NULL)
        printf("Stack Underflow!\n");
    else {
        struct Node* temp = top;
        printf("Popped element: %d\n", top->data);
        top = top->next;
        free(temp);
    }
}

void display() {
    if (top == NULL)
        printf("Stack is empty!\n");
    else {
        struct Node* temp = top;
        printf("Stack elements: ");
        while (temp != NULL) {
            printf("%d ", temp->data);
            temp = temp->next;
        }
        printf("\n");
    }
```

```
}

int main() {
    int choice;
    do {
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

Output:



3. Write a program to convert infix expression into postfix expression using stack.

```
#include <stdio.h>
#include <ctype.h>
#define MAX 100

char stack[MAX];
int top = -1;

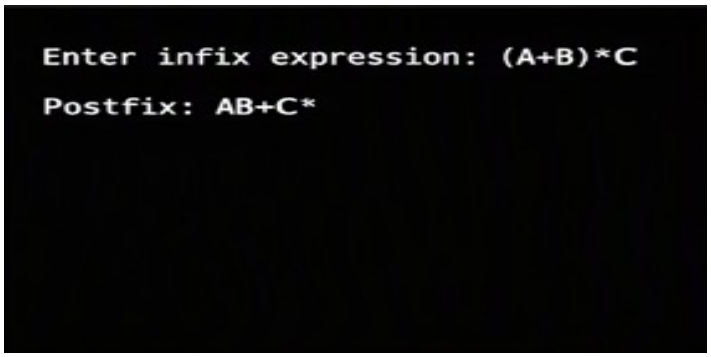
void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}

void infixToPostfix(char* exp) {
    char output[MAX]; int k = 0;
    for (int i = 0; exp[i]; i++) {
        if (isalnum(exp[i])) output[k++] = exp[i];
        else if (exp[i] == '(') push(exp[i]);
        else if (exp[i] == ')') {
            while (top != -1 && stack[top] != '(')
                output[k++] = pop();
            pop();
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(exp[i]))
                output[k++] = pop();
```

```
            push(exp[i]);
        }
    }
    while (top != -1) output[k++] = pop();
    output[k] = '\0';
    printf("Postfix: %s\n", output);
}

int main() {
    char exp[MAX];
    printf("Enter infix expression: ");
    scanf("%s", exp);
    infixToPostfix(exp);
    return 0;
}
```

Output:



4. Write a program to convert infix expression into prefix expression using stack.

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100
char stack[MAX]; int top = -1;
```

```
void push(char c) { stack[++top] = c; }
char pop() { return stack[top--]; }
int precedence(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return -1;
}

void reverse(char* exp) {
    int n = strlen(exp);
    for (int i = 0; i < n/2; i++) {
        char temp = exp[i];
        exp[i] = exp[n-i-1];
        exp[n-i-1] = temp;
    }
    for (int i = 0; i < n; i++) {
        if (exp[i] == '(') exp[i] = ')';
        else if (exp[i] == ')') exp[i] = '(';
    }
}

void infixToPrefix(char* exp) {
    reverse(exp);
    char output[MAX]; int k = 0;
    for (int i = 0; exp[i]; i++) {
        if (isalnum(exp[i])) output[k++] = exp[i];
        else if (exp[i] == '(') push(exp[i]);
```

```
        else if (exp[i] == ')') {
            while (top != -1 && stack[top] != '(')
                output[k++] = pop();
            pop();
        } else {
            while (top != -1 && precedence(stack[top]) >= precedence(exp[i]))
                output[k++] = pop();
            push(exp[i]);
        }
    }
    while (top != -1) output[k++] = pop();
    output[k] = '\0';
    reverse(output);
    printf("Prefix: %s\n", output);
}

int main() {
    char exp[MAX];
    printf("Enter infix expression: ");
    scanf("%s", exp);
    infixToPrefix(exp);
    return 0;
}
```

Output:



5. Write a program to evaluate a postfix expression using stack.

```
#include <stdio.h>

#include <ctype.h>

#define MAX 100


int stack[MAX]; int top = -1;


void push(int x) { stack[++top] = x; }

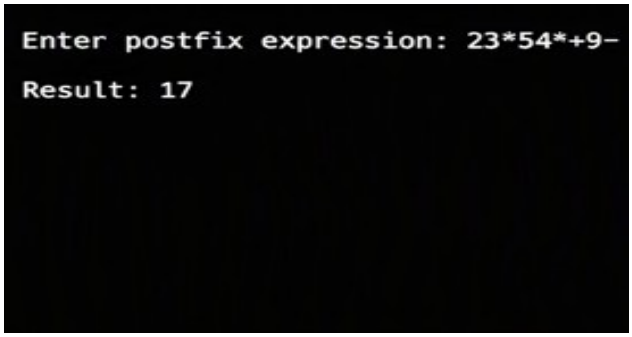
int pop() { return stack[top--]; }


int evaluatePostfix(char* exp) {
    for (int i = 0; exp[i]; i++) {
        if (isdigit(exp[i])) push(exp[i] - '0');
        else {
            int val2 = pop();
            int val1 = pop();
            switch (exp[i]) {
                case '+': push(val1 + val2); break;
                case '-': push(val1 - val2); break;
                case '*': push(val1 * val2); break;
                case '/': push(val1 / val2); break;
            }
        }
    }
```

```
    return pop();
}

int main() {
    char exp[MAX];
    printf("Enter postfix expression: ");
    scanf("%s", exp);
    printf("Result: %d\n", evaluatePostfix(exp));
    return 0;
}
```

Output:



LAB ASSIGNMENT 6

Queues

1. Write a menu driven program with 4 options (Insert, Delete, Display, and Exit) to demonstrate the working of Queues using arrays.

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void insert() {
    int val;
    if (rear == MAX - 1)
        printf("Queue Overflow!\n");
    else {
        printf("Enter value to insert: ");
        scanf("%d", &val);
        if (front == -1) front = 0;
        queue[++rear] = val;
    }
}

void delete() {
    if (front == -1 || front > rear)
        printf("Queue Underflow!\n");
    else
        printf("Deleted element: %d\n", queue[front++]);
}
```

```
void display() {
    if (front == -1 || front > rear)
        printf("Queue is empty!\n");
    else {
        printf("Queue elements: ");
        for (int i = front; i <= rear; i++)
            printf("%d ", queue[i]);
        printf("\n");
    }
}

int main() {
    int choice;
    do {
        printf("\n1.Insert\n2.Delete\n3.Display\n4.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

Output:

```
1.Insert
2.Delete
3.Display
4.Exit
Enter choice: 1
Enter value to insert: 10
Enter choice: 1
Enter value to insert: 20
Enter choice: 3
Queue elements: 10 20
Enter choice: 2
Deleted element: 10
Enter choice: 4
Exiting...
```

2. Write a menu driven program with 4 options (Insert, Delete, Display, and Exit) to demonstrate the working of Circular Queues (arrays.)

```
#include <stdio.h>

#define MAX 5


int queue[MAX];

int front = -1, rear = -1;


void insert() {
    int val;
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
        printf("Queue Overflow!\n");
    else {
        printf("Enter value to insert: ");
        scanf("%d", &val);
        if (front == -1)
            front = rear = 0;
        else if (rear == MAX - 1)
```

```
            rear = 0;
        else
            rear++;
        queue[rear] = val;
    }
}

void delete() {
    if (front == -1)
        printf("Queue Underflow!\n");
    else {
        printf("Deleted element: %d\n", queue[front]);
        if (front == rear)
            front = rear = -1;
        else if (front == MAX - 1)
            front = 0;
        else
            front++;
    }
}

void display() {
    if (front == -1)
        printf("Queue is empty!\n");
    else {
        printf("Queue elements: ");
        int i = front;
        while (1) {
            printf("%d ", queue[i]);
```

```
            if (i == rear) break;
            i = (i + 1) % MAX;
        }
        printf("\n");
    }
}

int main() {
    int choice;
    do {
        printf("\n1.Insert\n2.Delete\n3.Display\n4.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: printf("Exiting...\n"); break;
            default: printf("Invalid choice!\n");
        }
    } while (choice != 4);
    return 0;
}
```

Output:

```
1.Insert
2.Delete
3.Display
4.Exit
Enter choice: 1
Enter value to insert: 5
Enter choice: 1
Enter value to insert: 15
Enter choice: 1
Enter value to insert: 25
Enter choice: 3
Queue elements: 5 15 25
Enter choice: 2
Deleted element: 5
Enter choice: 1
Enter value to insert: 35
Enter choice: 3
Queue elements: 15 25 35
Enter choice: 4
Exiting...
```